



Università
degli Studi di
Messina

Data mining project Report

Course: Data Mining

Professor: Daniele Ravi

Student: Tairbek Akhayev, 551094

Table of Contents

1. Dataset exploration	3
1.1 Dataset overview	3
1.2 Data structure	4
1.3 Data visualization	4
Histogram Analysis	5
Boxplot Analysis	6
1.4 Relationship analysis	6
2. Data preprocessing	8
2.1 Handling missing values	8
2.2 Treating outliers	11
2.3 Encode categorical variables	13
2.4 Feature scaling and Normalization	14
3. EDA	16
3.1 Investigate the relationship between features and the target variable (Churn)	16
3.2 Further analysis	19
3.2.1 Correlation analysis	20
3.2.2 Correlation of Features with Churn (bar plot)	21
3.3. Feature Importance	22
3.3.1 Checking assumptions	24
3.4 Hypothesis test	29
4. Feature engineering	29
5. Modeling	30
5.1 Splitting the data into training and testing sets	30
5.2 Training models	31
5.3 Model evaluation	31
5.4 Cross validation	34
6. Model tuning	34
7. Model interpretation	35
7.1 LIME	36
8. Final model	37
8.1 Learning curve	38
9. Business Insights and Conclusion	40

1. Dataset exploration

1.1 Dataset overview

To begin this project, I imported the essential Python libraries that are widely used in data mining and machine learning tasks. These include:

- `pandas` and `NumPy` for handling and manipulating data efficiently.
- `seaborn` and `matplotlib` for creating visualizations to better understand patterns in the dataset.
- `Sklearn` for building and evaluating machine learning models.
- and `sys`, which can be useful for environment or runtime-related diagnostics.

These libraries provided the foundational tools needed to explore the dataset, visualize key trends, and eventually build predictive models.

```
import sys

# Scikit-Learn
import sklearn

# Common imports
import numpy as np
import pandas as pd
import numpy as np
import seaborn as sns
```

```
# To plot
import matplotlib as mpl
import matplotlib.pyplot as plt
```

After importing the necessary libraries, I loaded the dataset using `pandas`:

```
df = pd.read_csv('/Users/tairakhayev/Desktop/Data
Mining/project/dataset_churn.csv')
```

1.2 Data structure

To begin understanding the dataset, I first loaded and inspected its structure using basic exploratory commands such as `.shape`, `.head()`, `.info()`, `.describe()`, and `.nunique()`. This dataset consists of **3,749 entries** and **17 columns**, each representing a specific attribute related to customer demographics, service usage, billing information, and churn status.

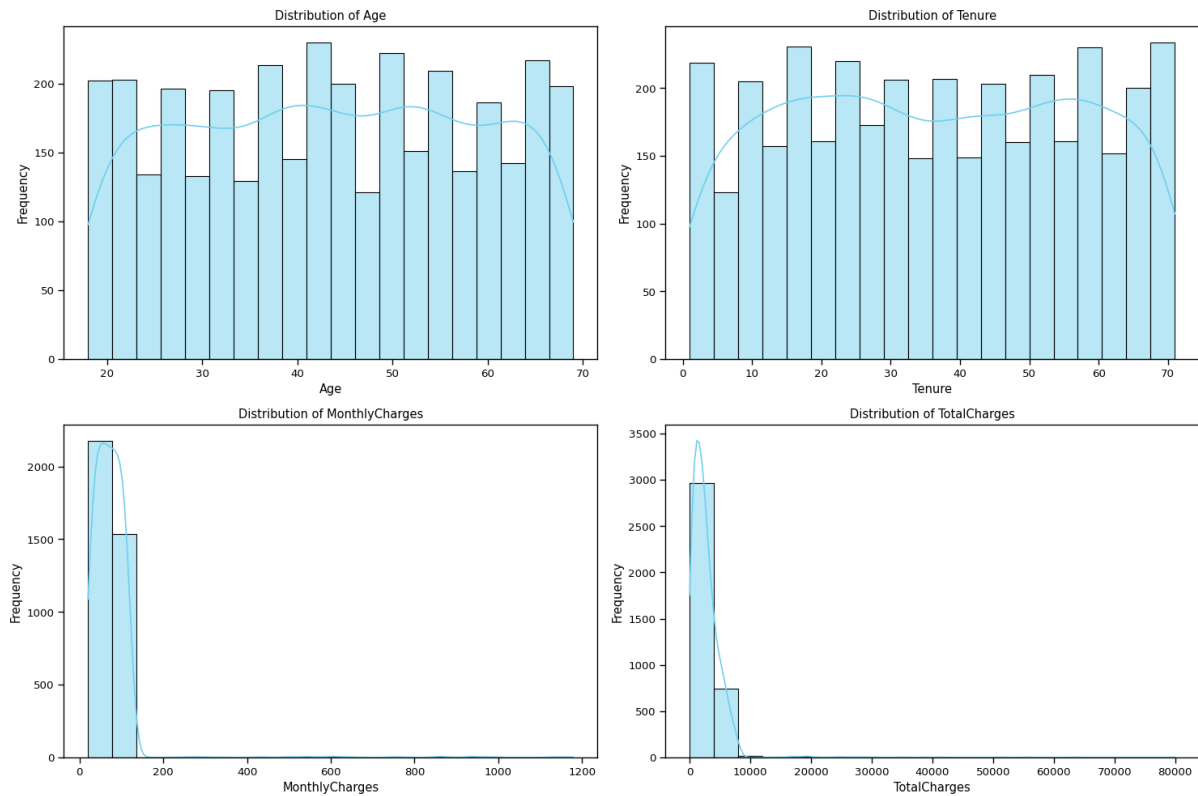
The dataset appears to describe customer behaviour in a telecom-like subscription service, where each row corresponds to a single customer. The target variable is **Churn**, which indicates whether the customer has left the service (Yes or No).

Several columns are categorical, such as `Gender`, `Contract`, `PaymentMethod`, and service-related features like `StreamingMusic`, `OnlineSecurity`, and `TechSupport`. Others, like `Age`, `Tenure`, `MonthlyCharges`, and `TotalCharges`, are numerical. Some columns (like `Age` and `Service_Internet`) have missing values, which are important to handle before modelling.

Statistical summaries using `describe()` show that variables like `MonthlyCharges` and `TotalCharges` have a wide range, with `TotalCharges` extending up to nearly 80,000, suggesting potentially high-value customers. Overall, the dataset provides a rich mix of numerical and categorical features, ideal for building a classification model to predict customer churn.

1.3 Data visualization

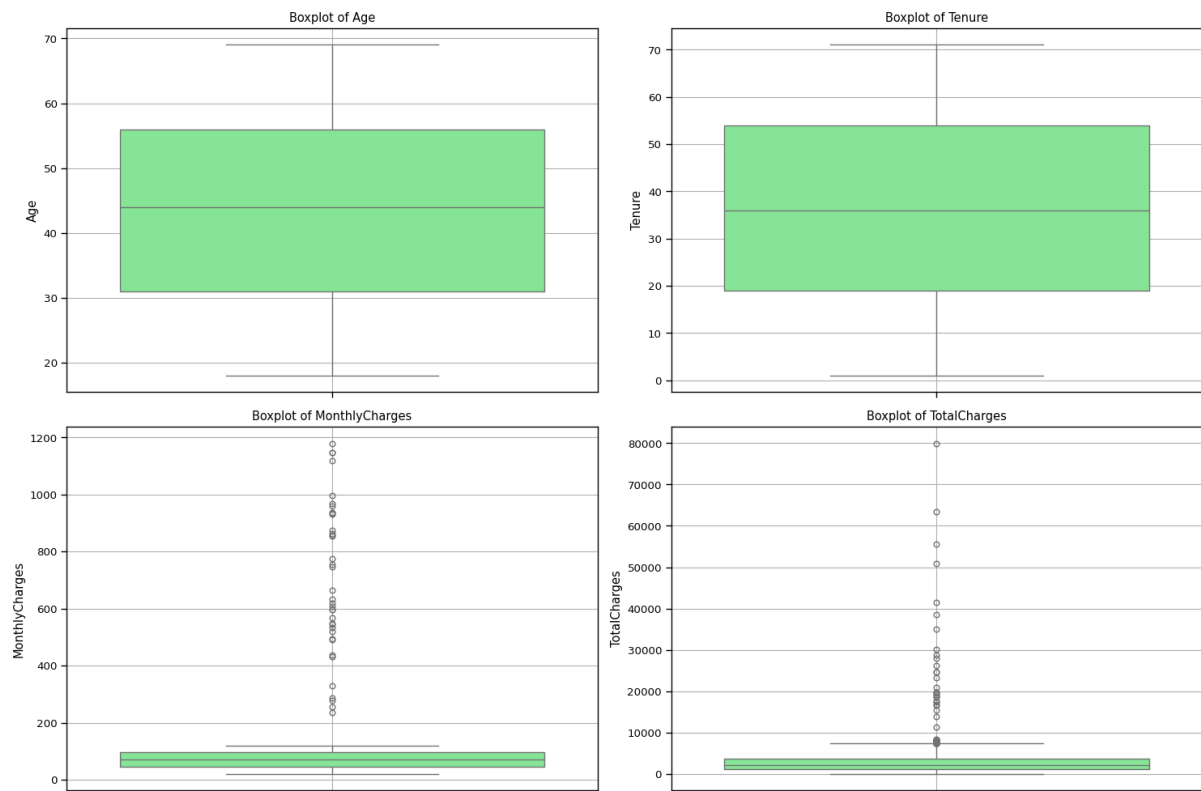
To better understand the distribution and behavior of the numerical features in the dataset, I visualized them using both **histograms** and **boxplots**.



Histogram Analysis

I did this to observe the shape and spread of each feature, because histograms with KDE curves help reveal the distribution pattern -whether it's symmetric, skewed, uniform, or multimodal.

- **Age** and **Tenure** appear relatively uniformly distributed across their ranges, suggesting that the dataset includes customers of various ages and durations of service.
- **MonthlyCharges** is slightly right skewed, but most values fall within a narrow range, indicating that most customers pay within a consistent price band.
- **TotalCharges** is strongly right skewed, with a long tail stretching toward high values. This makes sense because it's an accumulated amount, and customers with longer tenure naturally accumulate higher charges.



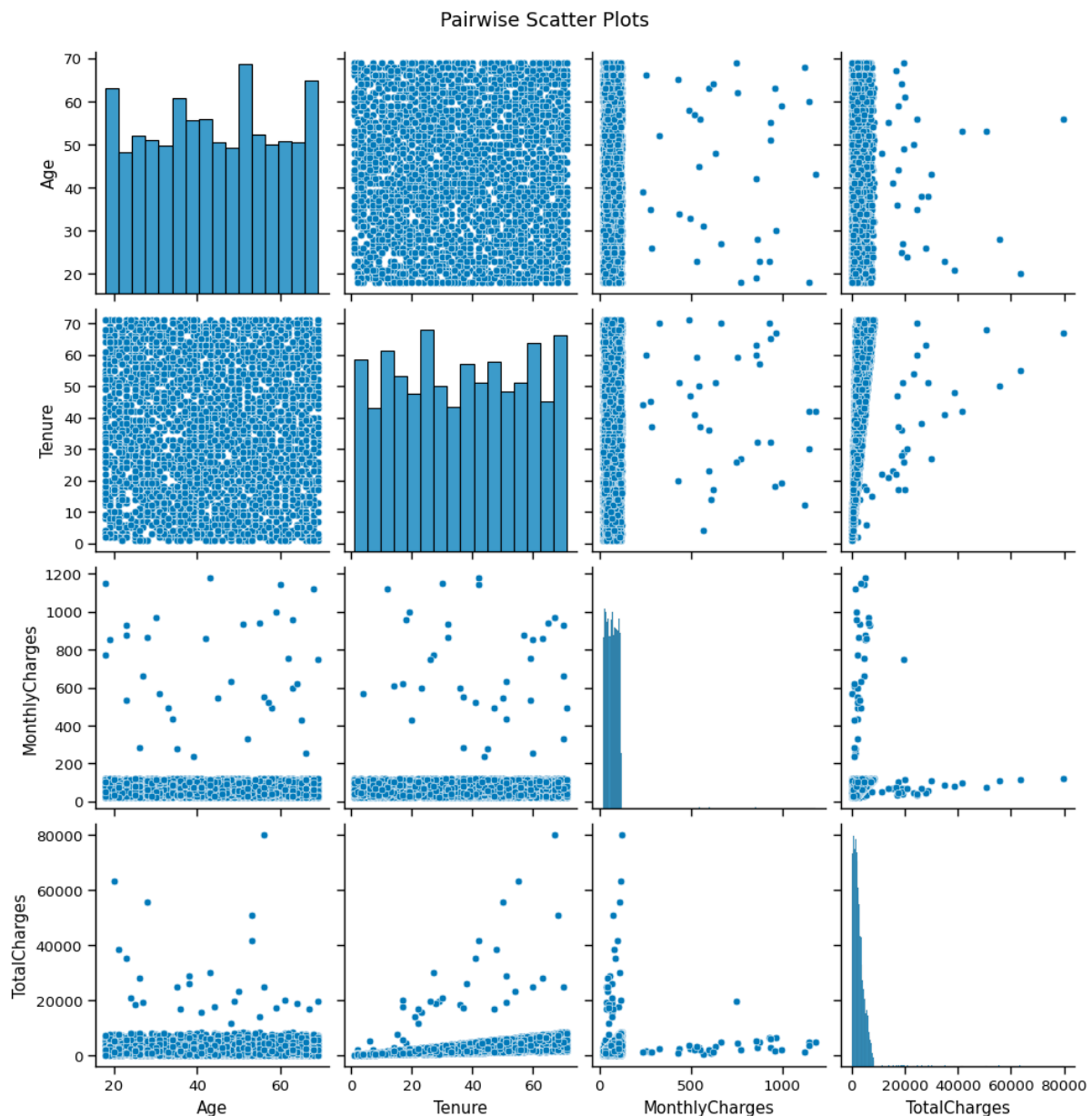
Boxplot Analysis

I did this to detect the presence of **outliers** and to get a visual sense of the **interquartile range (IQR)** and median values for each feature.

- **Age** and **Tenure** show very few outliers, and their distributions look balanced around the median.
- In contrast, both **MonthlyCharges** and especially **TotalCharges** contain a significant number of outliers. In the case of **TotalCharges**, we see extreme high-end values, some approaching or exceeding 70,000–80,000. These could correspond to long-term high-paying customers or data entry anomalies and may need to be handled or clipped before training models.

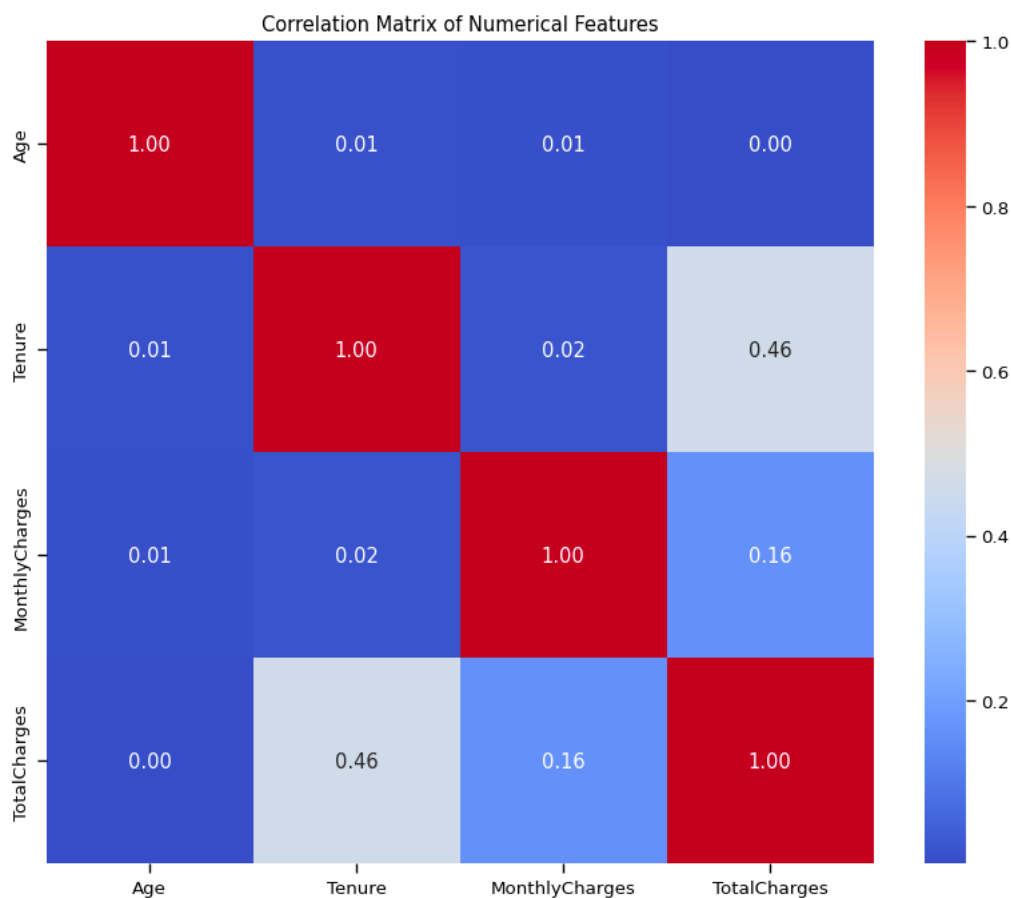
1.4 Relationship analysis

To better understand how the numerical features interact with one another, I created a **pairwise scatter plot matrix** and a **correlation heatmap**. I did this in order to visually and statistically examine potential linear relationships between key variables like **Age**, **Tenure**, **MonthlyCharges**, and **TotalCharges**, because identifying correlated features is important for feature selection and model behavior interpretation.



From the scatter plots, we can observe:

- There is **no clear relationship** between `Age` and any other feature — the scatter appears randomly distributed.
- A **strong positive trend** exists between `Tenure` and `TotalCharges`, which makes sense: the longer a customer stays, the more they pay in total.
- There's **no strong correlation** between `MonthlyCharges` and `Tenure` — this suggests that monthly fees are not directly tied to how long the customer has stayed.
- The relationship between `MonthlyCharges` and `TotalCharges` is somewhat positive, but more dispersed — possibly due to varying tenures among users with similar monthly rates.



From the heatmap:

- **Tenure and TotalCharges** have the highest correlation at **0.46**, confirming the visual trend seen in the scatter plot.
- **MonthlyCharges and TotalCharges** have a **moderate positive correlation (0.16)** — indicating some relationship, but with noise.
- All other correlations (especially involving **Age**) are **very weak (close to 0)**, suggesting that these features are **largely independent** from one another.

2. Data preprocessing

2.1 Handling missing values

Before proceeding with data modelling, I analysed the dataset for missing values to ensure data completeness and avoid errors during training.

Using `df.isnull().sum()` function we can see missing values:

- **Age:** 187 missing entries
- **PaymentMethod:** 187 missing entries
- **Service_Internet:** 721 missing entries

```
df = df.drop(columns=["Unnamed: 0"])
```

I dropped the column "Unnamed: 0" because it was simply an index column automatically added during the CSV export or import process. It did not contain any meaningful information for the analysis or the model. Keeping it would only add noise and unnecessary dimensionality to the dataset, so removing it helped keep the feature set clean and relevant.

```
from sklearn.impute import SimpleImputer
# Age: Impute missing values using mean or median imputation
age_imputer = SimpleImputer(strategy='mean')
df['Age'] = age_imputer.fit_transform(df[['Age']])
```

I imputed the missing values in the **Age** column using the **mean value** of the non-missing entries. I used the **mean** strategy instead of **median** because the distribution of the Age feature was approximately uniform and symmetric, without heavy skew or extreme outliers. In such cases, the mean provides a reliable central tendency and preserves more of the original variance in the data compared to the median.

```
# Service_Internet: Replace missing values with a new category
df['Service_Internet'].fillna('Unknown', inplace=True)
```

I filled the missing values in the **Service_Internet** column with the label '**Unknown**' because this is a categorical feature. Instead of removing rows or guessing the missing category, assigning 'Unknown' preserves the data and allows the model to treat this as a separate, learnable category. This is especially useful when the missingness itself might carry some information about customer behavior.

```
# PaymentMethod: Impute missing values using mode imputation
payment_imputer = SimpleImputer(strategy='most_frequent')
# Ensure that the input data is a 2D array
payment_data = df[['PaymentMethod']]
payment_data_imputed = payment_imputer.fit_transform(payment_data)
# Assign the imputed values back to the DataFrame
```

```
df['PaymentMethod'] = payment_data_imputed.reshape(-1) # Reshape to 1D array
```

I used **mode imputation** for the PaymentMethod column by applying

the “**most_frequent**” strategy. Since this column is categorical, it’s appropriate to replace missing values with the most common payment method among customers. This ensures the imputed values reflect realistic and frequent behavior patterns rather than introducing noise.

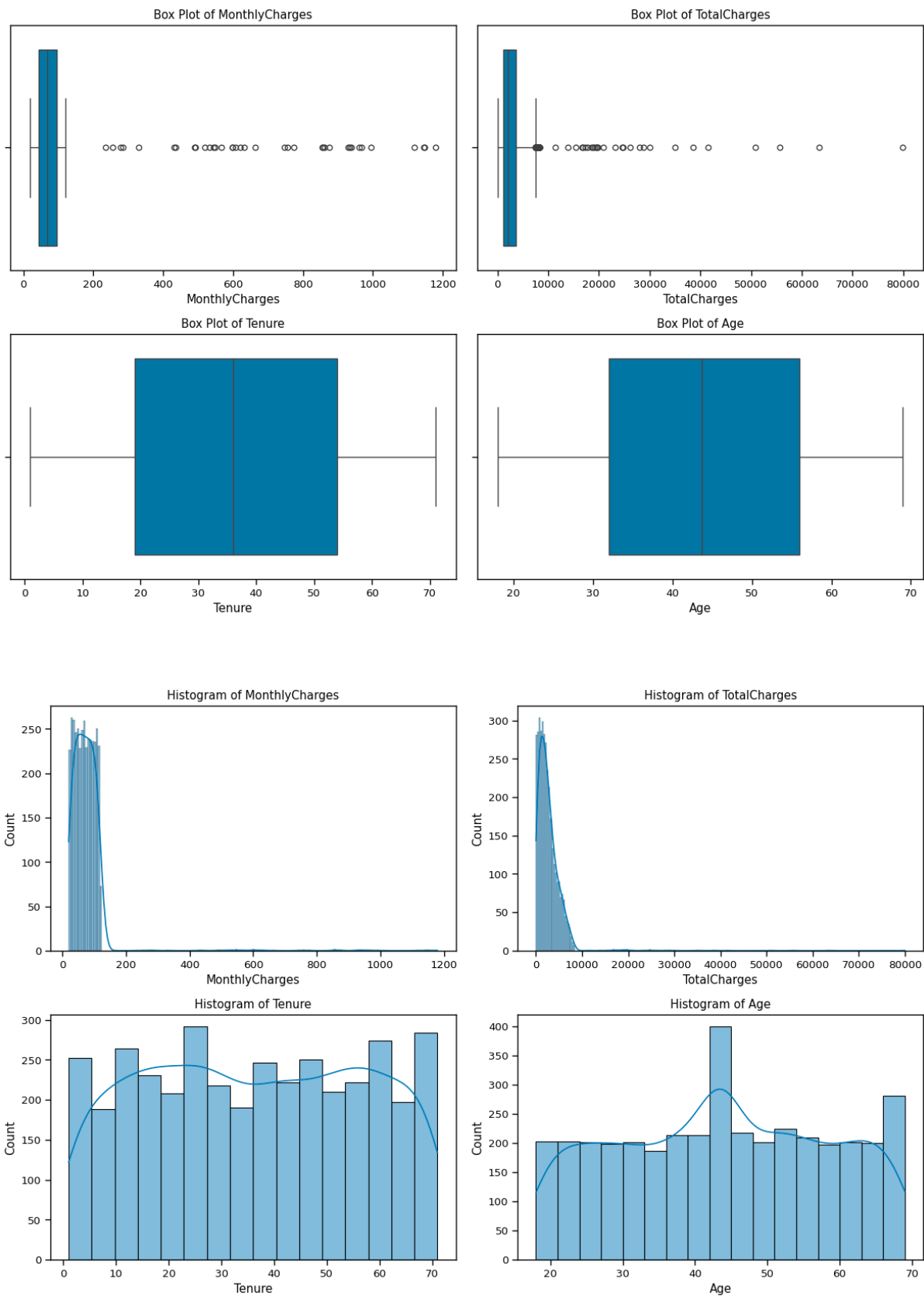
```
# Verify missing value handling
missing_values_after = df.isnull().sum()
print("Missing Values After Handling:")
print(missing_values_after)
```

I added this check to **verify that all missing values had been successfully handled** after the imputation steps.

This step is important because it confirms that the dataset is now clean and ready for further preprocessing or modelling. If any values were still missing at this point, I would need to go back and address them to avoid errors in the next steps of the pipeline.

```
Missing Values After Handling:
CustomerID      0
Age             0
Gender          0
Tenure          0
Service_Internet 0
Service_Phone   0
Service_TV      0
Contract        0
PaymentMethod   0
MonthlyCharges  0
TotalCharges    0
StreamingMovies 0
StreamingMusic  0
OnlineSecurity  0
TechSupport     0
Churn           0
dtype: int64
```

2.2 Treating outliers



As part of the initial data exploration, I visualized the main numerical features using box plots and histograms. This revealed that TotalCharges and MonthlyCharges have high variability and are right-skewed, with long tails and numerous outliers. These outliers could negatively impact statistical analysis and model performance.

To address this, I applied a **clipping strategy** based on the 5th and 95th percentiles. This allowed me to cap extreme values without removing any data, helping to reduce the influence of outliers while preserving the overall structure of the dataset.

```
from sklearn.cluster import KMeans

# Select numerical features for clustering
numerical_features = ['MonthlyCharges', 'TotalCharges']
X = df[numerical_features]

# Fit K-Means clustering
kmeans = KMeans(n_clusters=2) # Assuming there are outliers in one cluster
kmeans.fit(X)

# Get cluster labels and centroids
cluster_labels = kmeans.labels_
centroids = kmeans.cluster_centers_

# Calculate distance from each point to its cluster centroid
distances = []
for i in range(len(X)):
    cluster_idx = cluster_labels[i]
    centroid = centroids[cluster_idx]
    distance = np.linalg.norm(X.iloc[i] - centroid)
    distances.append(distance)

# Identify outliers as points with distances above a certain threshold
outlier_threshold = np.percentile(distances, 95) # Adjust percentile as needed
outliers = df[distances > outlier_threshold]

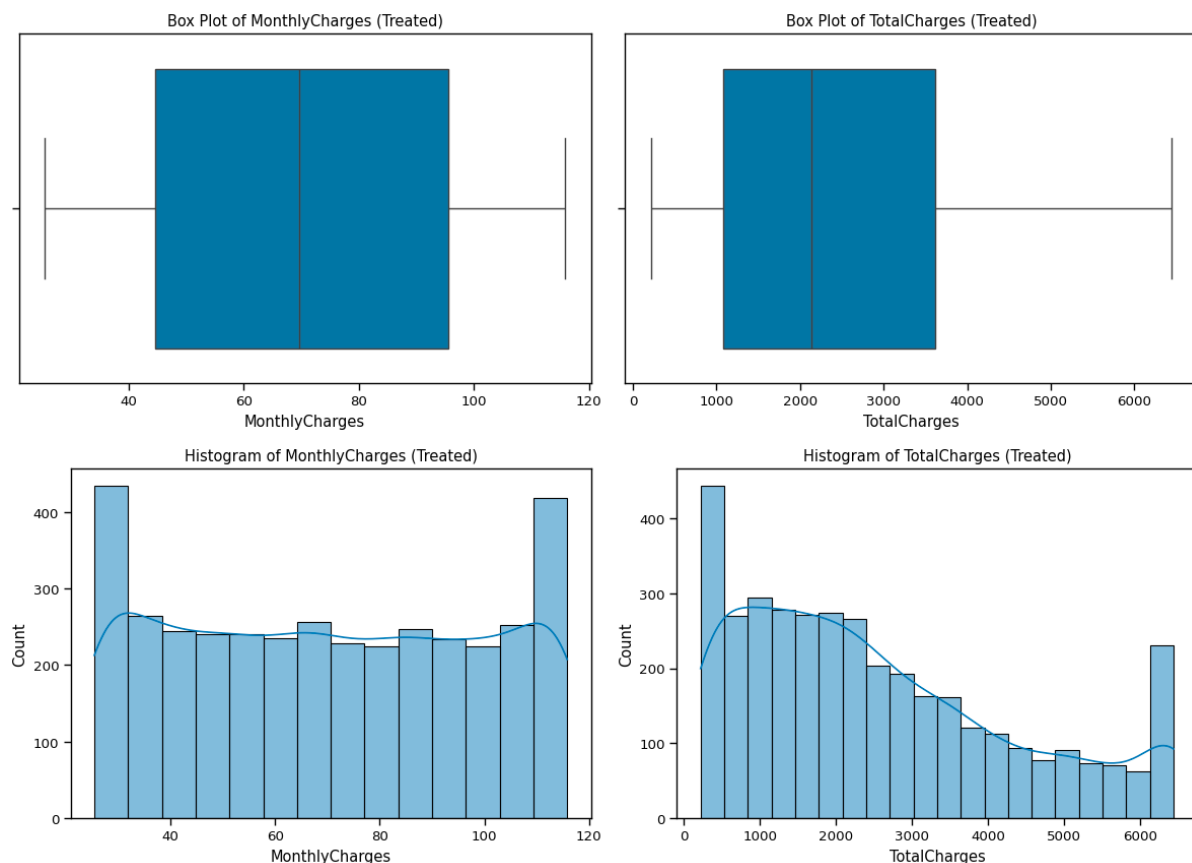
print("Outliers identified using K-Means Clustering:")
print(outliers)
```

I applied **K-Means clustering** as an additional method to detect outliers in the dataset. I did this to identify customers whose MonthlyCharges and TotalCharges are unusually far from typical spending patterns, because these two features showed high skewness and variability.

By clustering the data into two groups and measuring the **distance of each point from its cluster centroid**, I was able to flag potential outliers — specifically, those falling in the top 5% of distances. This method provided a more data-driven way to detect unusual patterns, complementing the earlier boxplot-based approach.

Using clustering in this way helps surface **extreme cases** that might distort model training, especially when numerical ranges vary greatly.

I applied **clipping** to the features MonthlyCharges and TotalCharges to treat outliers without removing data. I did this because both features showed extreme values that could distort model training and reduce generalization performance.



After clipping, I visualized the updated distributions using box plots and histograms. The plots showed that the outlier values were successfully compressed into a more compact range, while the overall shape and information in the distribution remained intact. This confirmed that the treatment helped balance the data without significant information loss.

2.3 Encode categorical variables

I encoded the binary categorical columns `OnlineSecurity`, `TechSupport`, and `Churn` by mapping 'Yes' to `True` and 'No' to `False`. I did this to prepare these features for modelling, because working with Boolean values (`True/False`) or their numeric equivalents (`1/0`) is necessary for most machine learning algorithms.

Using this approach also preserved the semantic meaning of the features:

- `True` = service is active, or the customer churned,
- `False` = service is not active, or the customer stayed.

This step ensures consistency in the dataset and simplifies further transformations like converting Booleans to integers (if needed) or feeding them directly into models that support Boolean input.

```
# Define the columns to map
columns_to_map = ['OnlineSecurity', 'TechSupport', 'Churn']

# Define the mapping dictionary
mapping_dict = {'No': False, 'Yes': True}

# Map 'No' and 'Yes' to 0 and 1 for the specified columns
df[columns_to_map] = df[columns_to_map].replace(mapping_dict)

# Display the updated DataFrame
df.head()
```

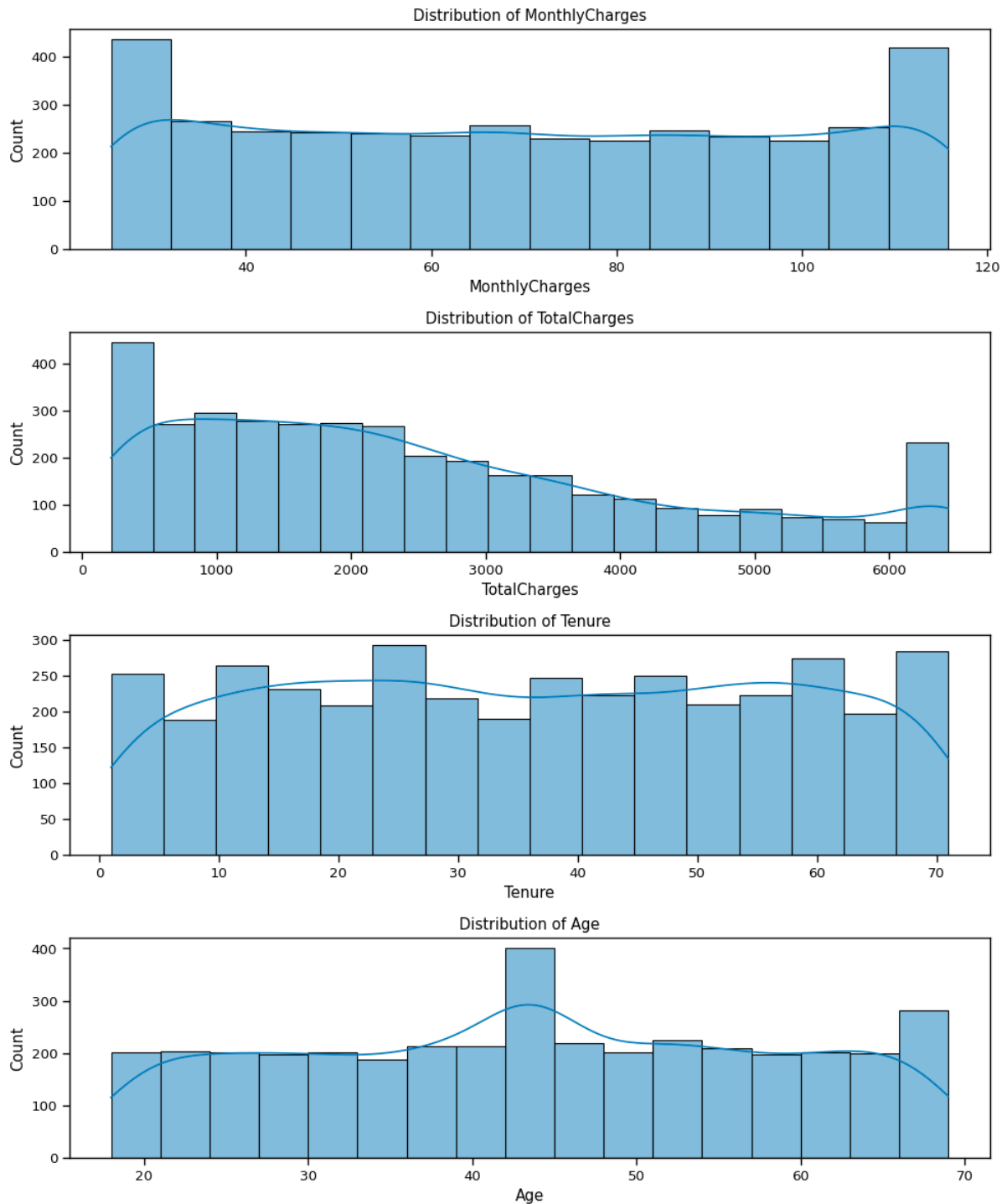
I applied **one-hot encoding** to the categorical columns Contract and PaymentMethod. I did this to convert each unique category within these columns into its own binary (0 or 1) feature, because machine learning models require numeric input and cannot directly process string-based categories.

```
# Perform one-hot encoding for categorical variables
columns_to_encode = ['Contract', 'PaymentMethod']
df = pd.get_dummies(df, columns=columns_to_encode)

df.head()
```

2.4 Feature scaling and Normalization

Before applying feature scaling, I first examined the histograms of all numerical features to understand their raw distributions after handling missing values. This helped assess whether scaling was necessary.



I applied Min-Max Scaling to the selected numerical features to bring them to a common scale between 0 and 1. This step was essential because many machine learning algorithms, particularly those that rely on distance calculations such as K-Nearest Neighbors (KNN) or Support Vector Machines (SVM), are sensitive to the range of input features. Without normalization, features with larger scales could dominate others and bias the model. Min-Max Scaling was chosen specifically because it rescales the data without altering the original distribution, and since outliers had already been addressed earlier in the pipeline, it provided

an effective and straightforward solution. By applying this technique to variables such as Age, Tenure, MonthlyCharges, and TotalCharges, I ensured that all features contributed equally during the training phase and avoided any imbalance due to differing value ranges.

```
from sklearn.preprocessing import MinMaxScaler

# Min-Max Scaling
scaler = MinMaxScaler()
df[numerical_features] = scaler.fit_transform(df[numerical_features])
```

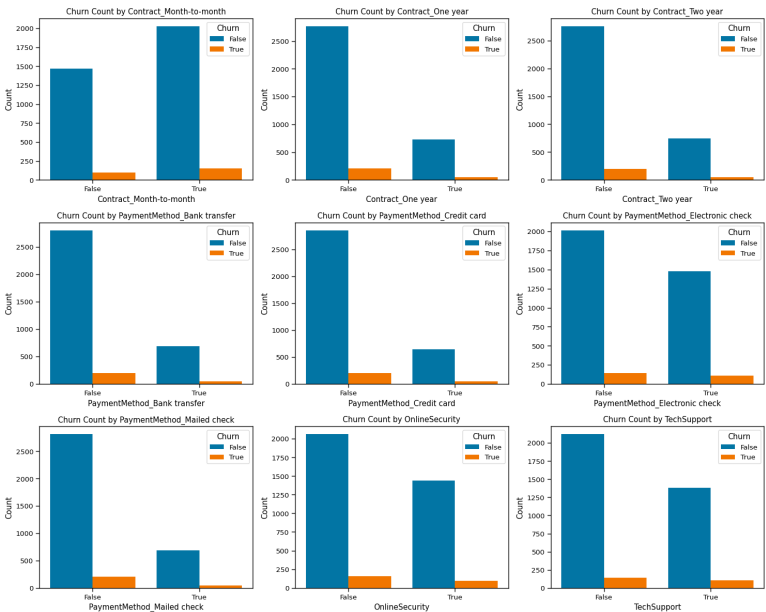
I saved the final cleaned DataFrame to a CSV file named **cleaned_dataset.csv**. This file contains only the processed data, with the index column removed for simplicity.

CustomerID	Age	Gender	Tenure	Service_Intel	Service_Phor	Service_TV	MonthlyChar	TotalCharges	StreamingMc	StreamingMl	OnlineSecuri	TechSupport	Churn	Contract_Mc	Contract_On	Contract_Tw	PaymentMet	PaymentMet	PaymentMet	PaymentMet
08729464-bc	0.74509804	Male	0.17142857	DSL	Yes	No	0.51379761	0.11469843	No	No	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	TRUE
af95bc95-ba	1	Male	0.17142857	DSL	No	Yes	0.94653566	0.19775231	Yes	Yes	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE
1fe7eee6-22	0.54901961	Male	0.84285714	Fiber optic	No	Yes	1	1	Yes	Yes	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE
f736fe7b-1b4	0.2745098	Female	0.8	Fiber optic	Yes	Yes	0.58328354	0.68030725	No	Yes	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE

3. EDA

3.1 Investigate the relationship between features and the target variable (Churn)

To understand how various features relate to customer churn, I conducted an exploratory analysis using bar plots, box plots, and a correlation heatmap. This step helps identify the strongest churn indicators and guides feature selection for modeling.



Bar plots showed churn counts for different contract types, payment methods, and service-related features:

- **Contract Type:**

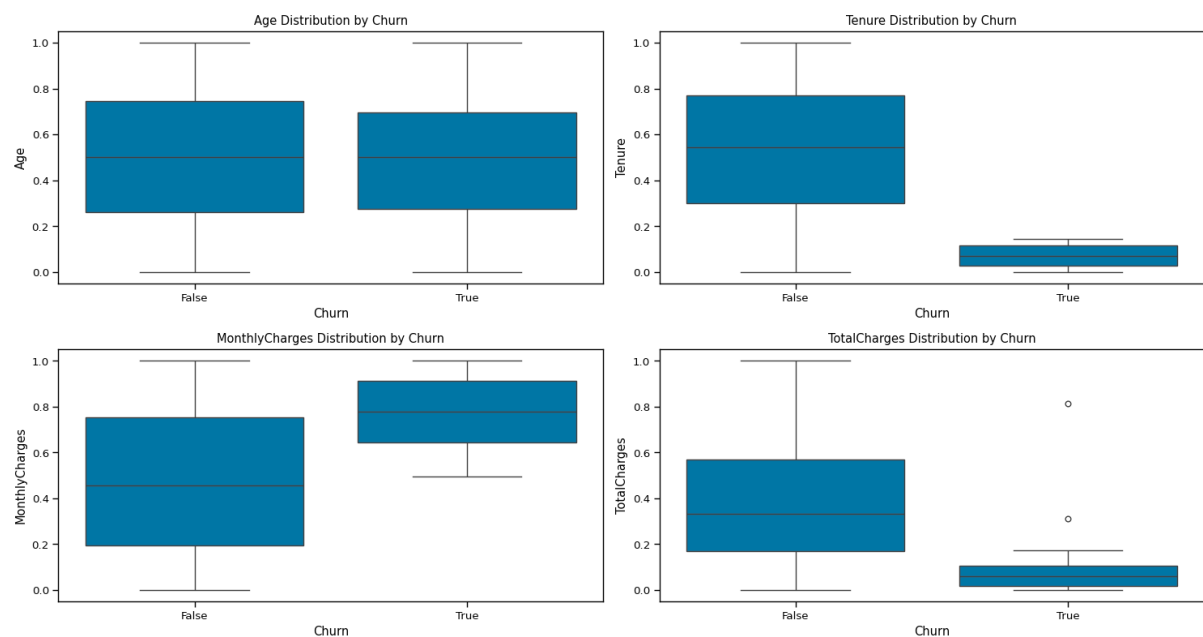
Customers with **month-to-month contracts** have a noticeably higher churn rate compared to those on one- or two-year contracts. Longer contracts are associated with greater customer retention.

- **Payment Method:**

The churn rate is highest among customers who pay by **electronic check**, suggesting that this group might be more price-sensitive or less committed. Those using credit cards or bank transfers tend to churn less.

- **OnlineSecurity and TechSupport:**

Customers who **do not have** these services are more likely to churn, implying that value-added services contribute to customer retention.



Box plots compared distributions of numerical features for churned and non-churned customers:

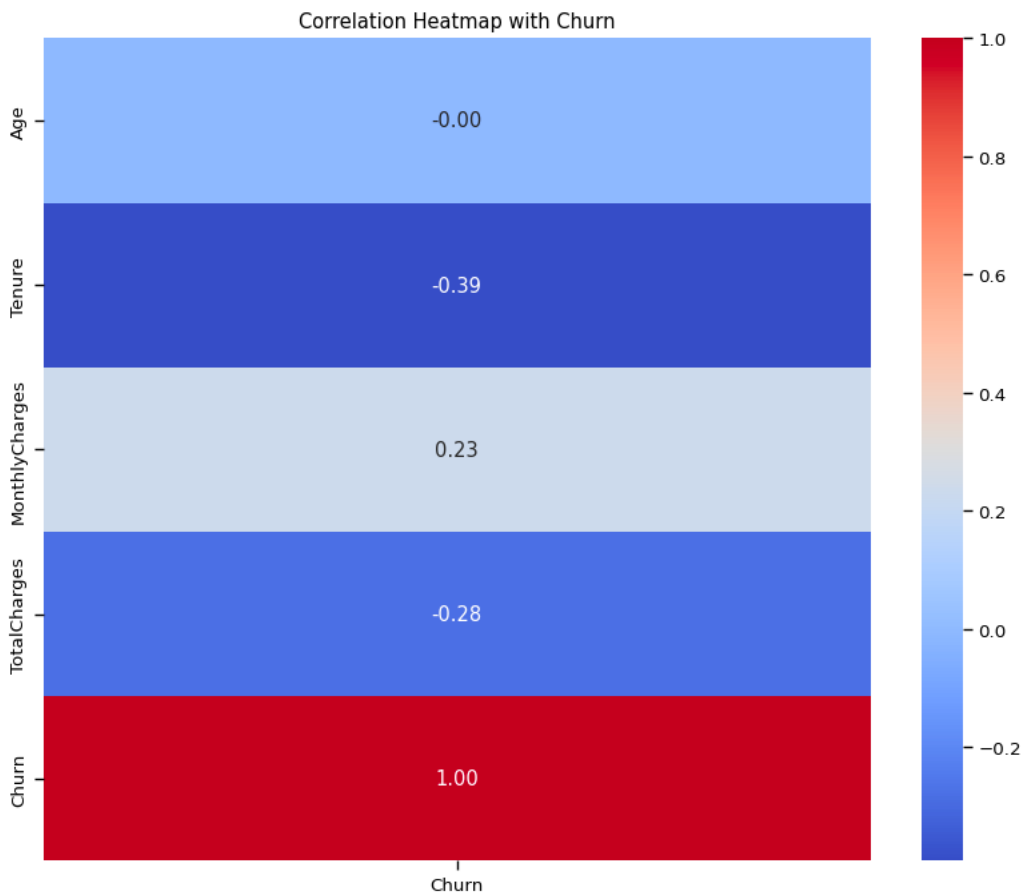
- **Tenure:**

Churned customers typically have **shorter tenure**, reinforcing that newer users are more likely to leave.

- **MonthlyCharges:**

Those who churn tend to have **higher monthly charges**, indicating a potential price-related churn pattern.

- **TotalCharges:**
non-churned customers generally have **higher total charges**, which makes sense since they've been around longer.
- **Age:**
The distribution of age does not show a strong difference between churned and non-churned customers.



The correlation heatmap quantifies linear relationships between numerical features and churn:

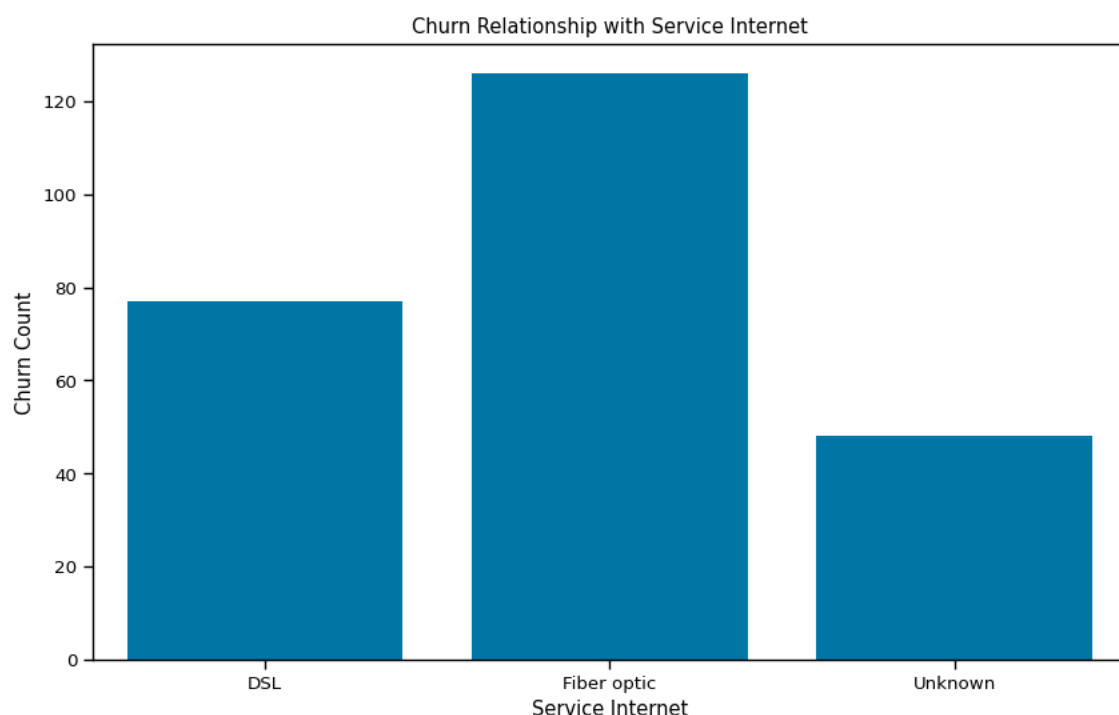
- **Tenure** has a moderate **negative correlation (-0.39)** with churn — longer-tenured customers are less likely to churn.
- **TotalCharges** also shows a **negative correlation (-0.28)**, aligned with the earlier box plot insights.
- **MonthlyCharges** has a **positive correlation (0.23)** with churn, suggesting that higher monthly fees might increase churn risk.
- **Age** has almost **no correlation (≈ 0)** with churn.

3.2 Further analysis

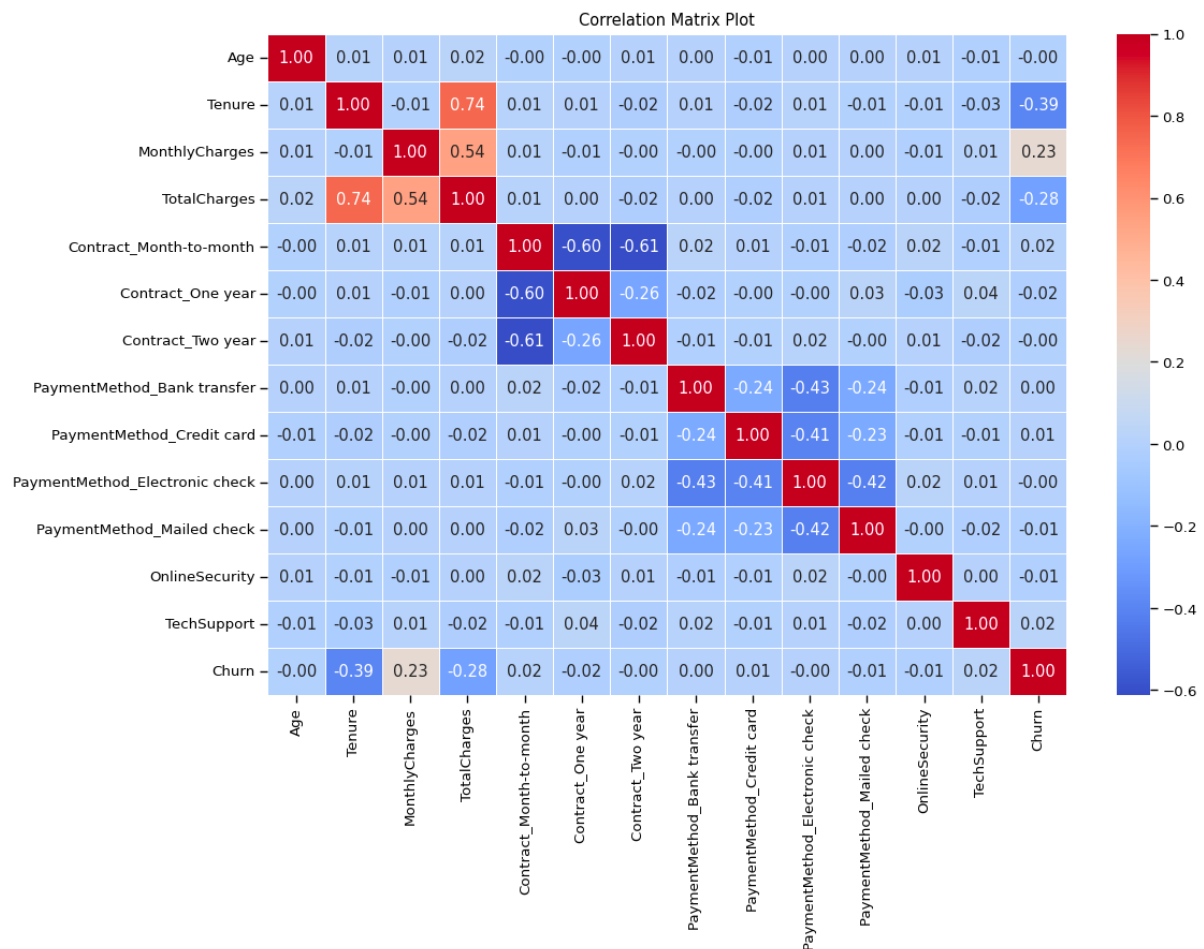
To explore the relationship between internet service types and customer churn, I first used the `unique()` function to identify the distinct values in the `Service_Internet` column. This revealed the different service options offered by the company, such as DSL, Fiber optic, and Unknown. I then visualized the churn counts for each category using a bar plot created with the Seaborn library.

The plot allowed for a clear comparison of churn rates across the different internet service types, highlighting any significant differences. Additional visual adjustments such as axis labels, a title, and figure size were made to enhance readability.

Interestingly, when reviewing the overall distribution of service types, I found that the number of customers using **Fiber optic** was higher than those using **DSL**, and **DSL** was more common than the **Unknown** category. This imbalance should be considered when interpreting the results, and the `Service_Internet` feature was therefore excluded from further modelling to avoid misleading influence in the analysis.



3.2.1 Correlation analysis



To better understand the linear relationships between features and the target variable, a correlation matrix was generated. This matrix included both numerical features and the one-hot encoded categorical variables.

The results revealed that Tenure had the strongest correlation with Churn at -0.39 , indicating that customers who had been with the company longer were less likely to churn. Similarly, TotalCharges showed a negative correlation of -0.28 , which is consistent with the fact that long-term customers tend to accumulate higher charges and are more likely to stay.

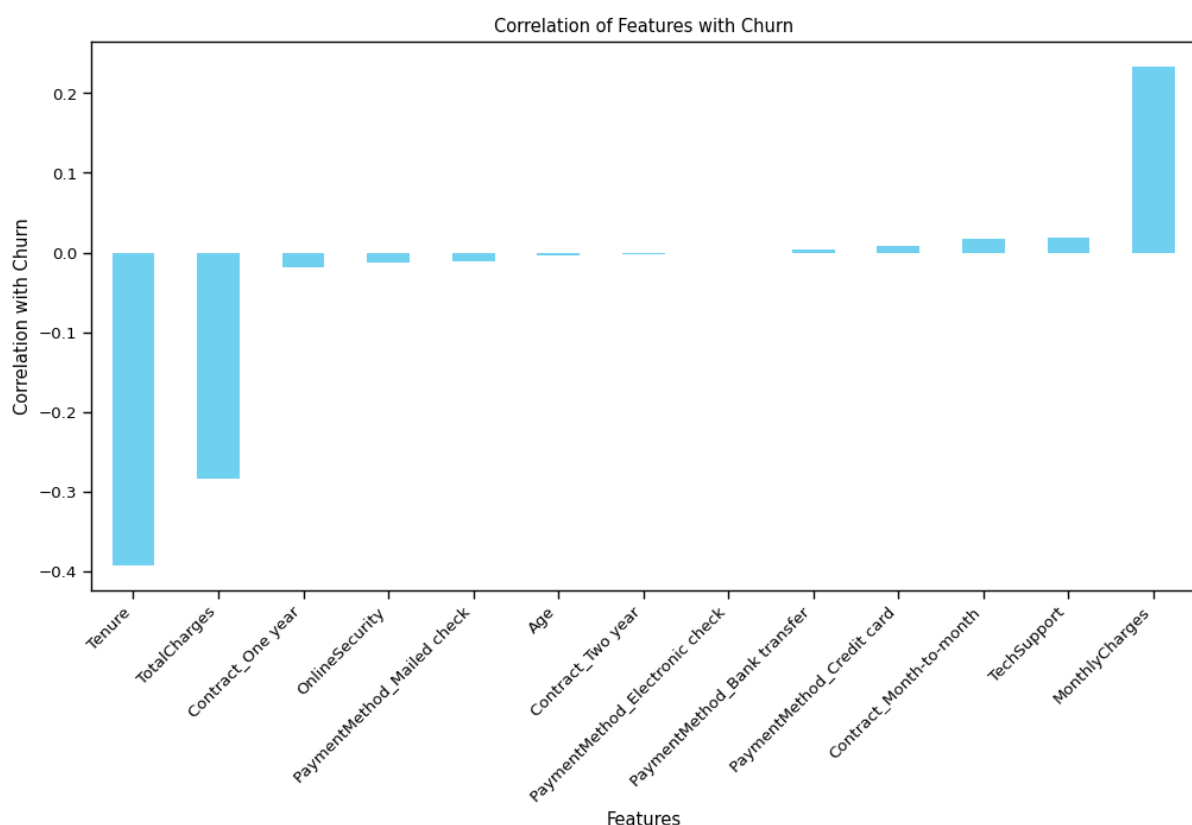
On the other hand, MonthlyCharges exhibited a positive correlation of 0.23 with churn, suggesting that customers with higher monthly payments were more prone to leaving the service.

Contract types also showed meaningful relationships:

Contract_Month-to-month was positively correlated with churn, while Contract_One year and Contract_Two year had slightly negative correlations, reinforcing the observation that customers on longer contracts are more likely to remain. Features such as OnlineSecurity and TechSupport showed weak negative correlations with churn, suggesting that access to support services might help with customer retention. Meanwhile, Age had virtually no correlation with churn, implying that age is not a strong predictor in this context.

In summary, this correlation analysis helped highlight the most relevant features for modeling churn, particularly tenure, charges, and contract type. These insights informed feature selection for subsequent model development.

3.2.2 Correlation of Features with Churn (bar plot)



To further assess the predictive relevance of individual features, I computed the Pearson correlation between each variable and the target Churn. The results were visualized in a bar chart, allowing for a direct comparison of how strongly each feature is linearly associated with customer churn.

The analysis revealed that **Tenure** had the strongest negative correlation with churn (-0.39), indicating that customers with longer service durations are significantly less likely to leave. Similarly, **TotalCharges** exhibited a negative correlation (-0.28), consistent with the idea that customers who have invested more over time tend to remain subscribed.

In contrast, **MonthlyCharges** showed a moderate **positive correlation** (0.24), suggesting that customers with higher monthly fees are more prone to churn—potentially due to cost-related dissatisfaction. Additionally, **Contract_Month-to-month** had a slight positive correlation, supporting the notion that customers without long-term commitments are more likely to discontinue service.

Other features, including payment methods, age, and support services (e.g., OnlineSecurity, TechSupport), demonstrated relatively weak or negligible correlations with churn when considered in isolation. While they may still contribute value in combination with other variables, their standalone predictive power is limited.

Overall, this correlation analysis highlights **Tenure**, **TotalCharges**, **MonthlyCharges**, and contract type as key drivers of churn, which will be prioritized in the modelling phase.

3.3. Feature Importance

```
Feature Importance Scores:
MonthlyCharges      0.390877
Tenure              0.360458
TotalCharges        0.204682
Age                 0.023104
TechSupport         0.003467
OnlineSecurity      0.003228
PaymentMethod_Electronic check 0.002361
Contract_Month-to-month 0.002291
PaymentMethod_Mailed check 0.002172
Contract_Two year   0.001966
PaymentMethod_Bank transfer 0.001947
PaymentMethod_Credit card 0.001846
Contract_One year   0.001601
dtype: float64
```

To evaluate the relative contribution of each feature in predicting customer churn, I trained a **Random Forest classifier** and extracted the feature importance scores. This approach helps identify which variables the model relies on most when making decisions.

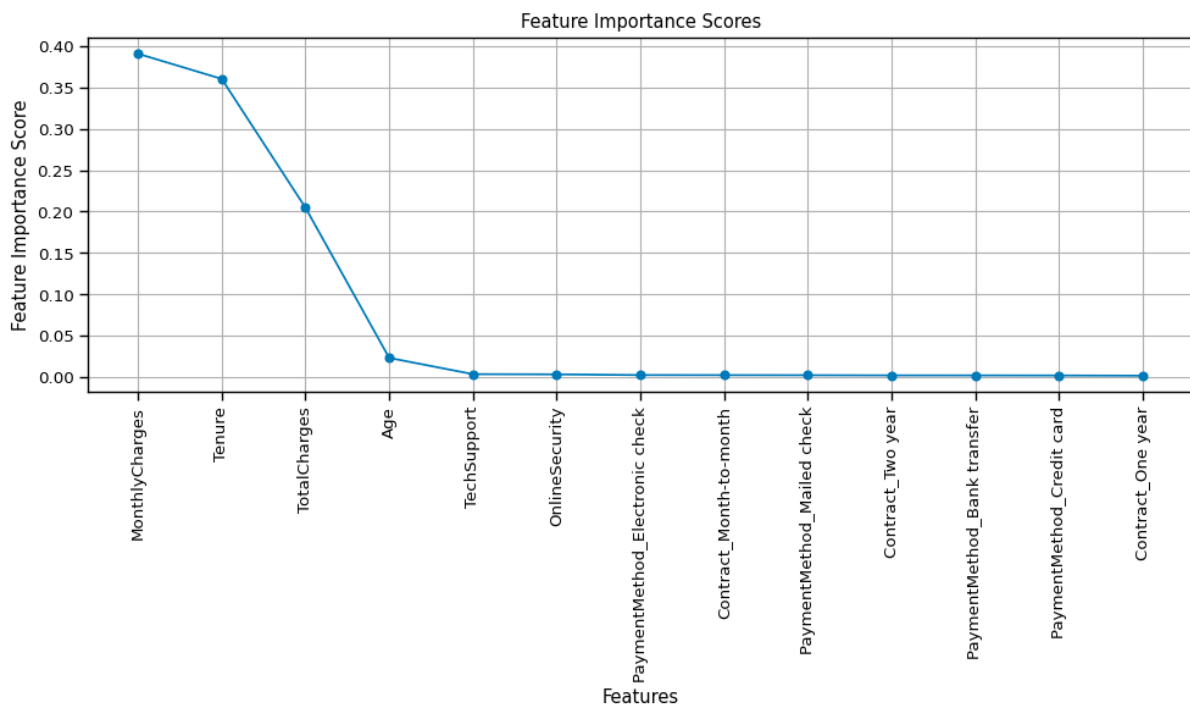
The results revealed that the top three most influential features were:

- **MonthlyCharges**, contributing approximately **39%** of the total decision power,
- **Tenure**, accounting for around **36%**, and
- **TotalCharges**, with an importance of roughly **20%**.

These findings confirm earlier correlation results, highlighting that customers who pay more per month or have been subscribed for a shorter period are more likely to churn.

Other features, such as Age, TechSupport, OnlineSecurity, and various contract or payment method encodings, showed **minimal impact** on the model's decision-making. While these may still contribute through interaction effects, their individual predictive power is relatively weak.

In summary, this analysis reinforces the significance of **monthly billing amount**, **customer tenure**, and **total spending** as the most critical variables influencing churn behaviour. These features will be prioritized in subsequent modelling and optimization phases.



I visualized the **feature importance scores** derived from the trained Random Forest model. The plot displays the ranked importance of each feature, with higher scores indicating greater influence on the model's decision-making process.

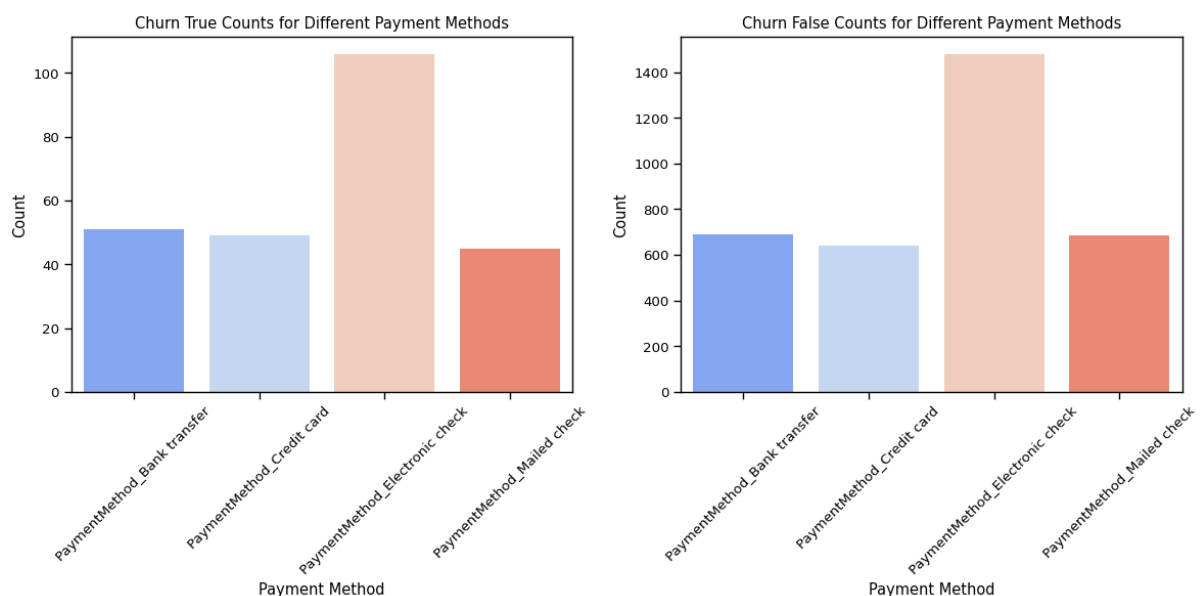
The chart clearly shows that **MonthlyCharges**, **Tenure**, and **TotalCharges** are by far the most significant predictors of churn. These three features collectively dominate the model's performance, confirming their strong predictive value observed in previous correlation and EDA analyses.

Other features such as Age, TechSupport, OnlineSecurity, and various encoded payment and contract types show **minimal impact**, with importance scores close to zero. While these may still play a role in capturing interaction effects, their standalone contribution to the model is limited.

This visualization reinforces the idea that customer billing behaviour and subscription duration are central to understanding churn. Therefore, these top-ranked features should be prioritized during further modelling, optimization, and deployment phases.

3.3.1 Checking assumptions

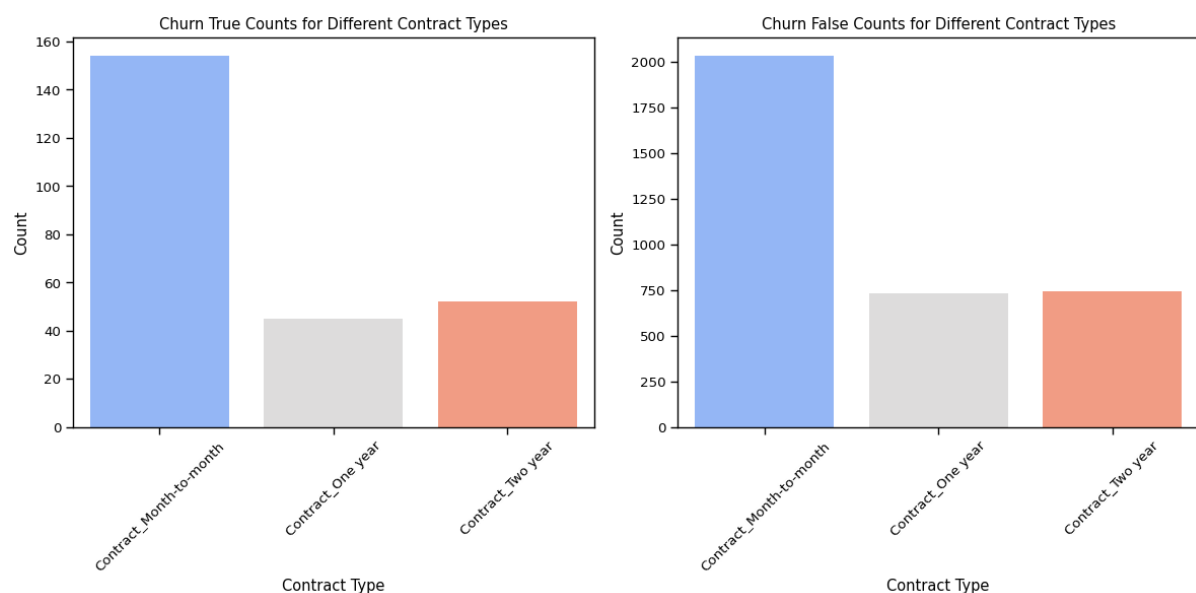
To examine the impact of payment method on churn, I split the data into churned and non-churned groups and visualized the counts for each payment method. While Electronic Check users showed a slightly higher churn rate, this method was also common among retained customers. Overall, the differences between methods were minor, suggesting that payment type has limited predictive value compared to stronger features like tenure or monthly charges. Nonetheless, the slight trend may still inform targeted retention efforts.



To examine the effect of contract duration on customer churn, I compared the distribution of churned and non-churned customers across different contract types. The data was segmented into two groups based on the Churn variable, and the counts for each contract type were visualized using bar plots.

The results show that customers on month-to-month contracts are significantly more likely to churn than those on one-year or two-year contracts. This aligns with expectations, as shorter commitments offer customers more flexibility to cancel services at any time. In contrast, churn rates are noticeably lower among customers with longer contracts, suggesting that contract length has a strong inverse relationship with churn.

This finding reinforces earlier correlation and feature importance results and highlights contract type as a key feature for predicting customer retention.

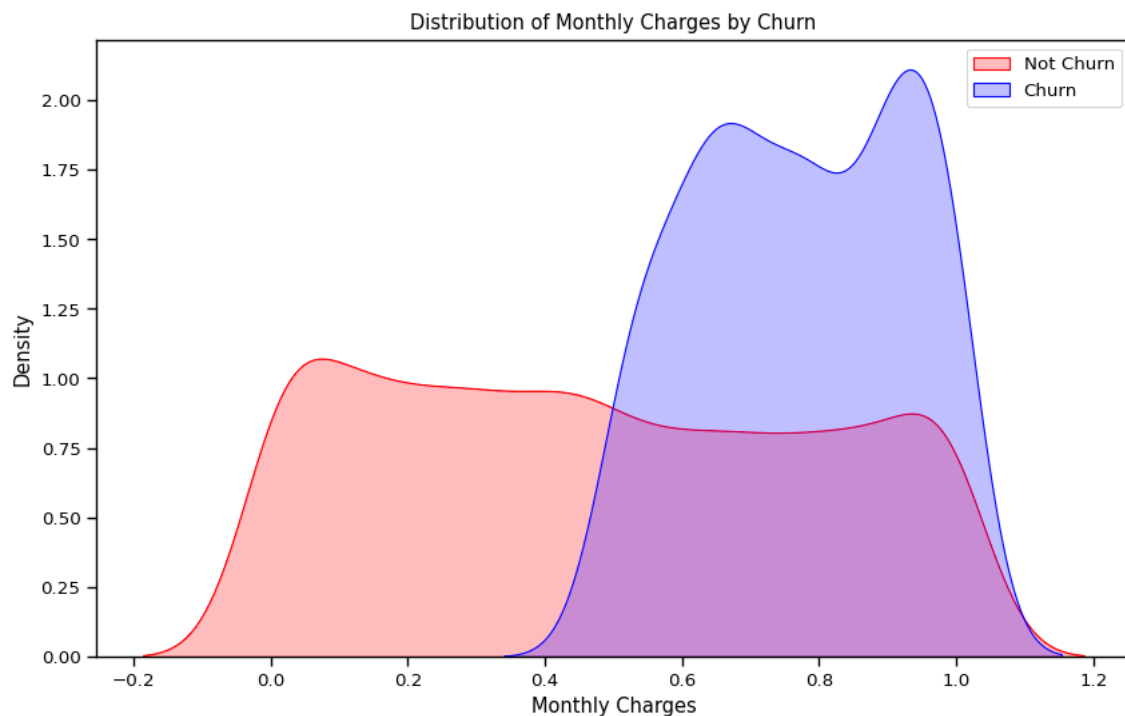


To examine how monthly billing amounts relate to churn behavior, I plotted kernel density estimates (KDE) for MonthlyCharges, segmented by churn status. This visualization allows for a direct comparison of the distribution of monthly charges between customers who stayed and those who left.

The plot reveals a clear distinction between the two groups. Customers who churned tend to have higher monthly charges, as evidenced by the peak of the blue curve being shifted to the right. In contrast, non-churned customers are more concentrated in the lower billing range.

This observation supports earlier correlation and feature importance findings, which identified MonthlyCharges as one of the most influential features for predicting churn. High

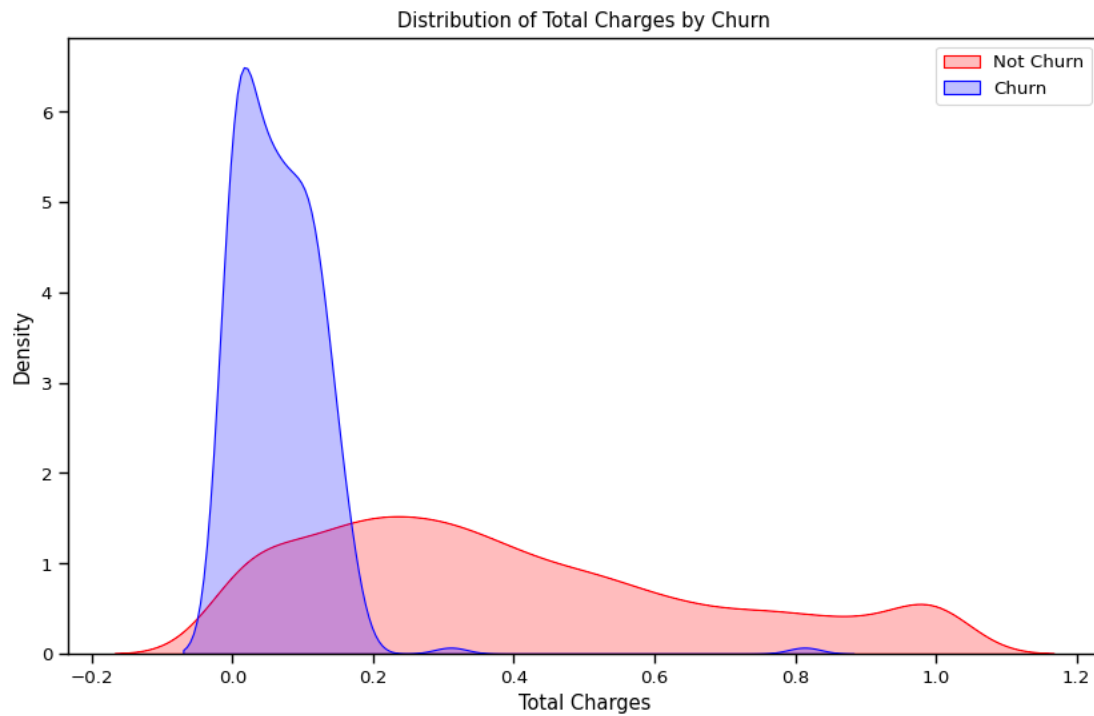
monthly costs may reflect customer dissatisfaction with value or affordability, making this an important variable for designing retention strategies.



To analyze the relationship between total customer spending and churn behavior, I plotted a kernel density estimate (KDE) of the TotalCharges variable for churned and non-churned customers.

The visualization shows a distinct difference between the two groups. Customers who churned are heavily concentrated in the lower range of total charges, while those who stayed exhibit a broader and more right-skewed distribution, indicating higher accumulated spending. This suggests that long-term, higher-spending customers are less likely to leave, likely due to greater investment and satisfaction with the service.

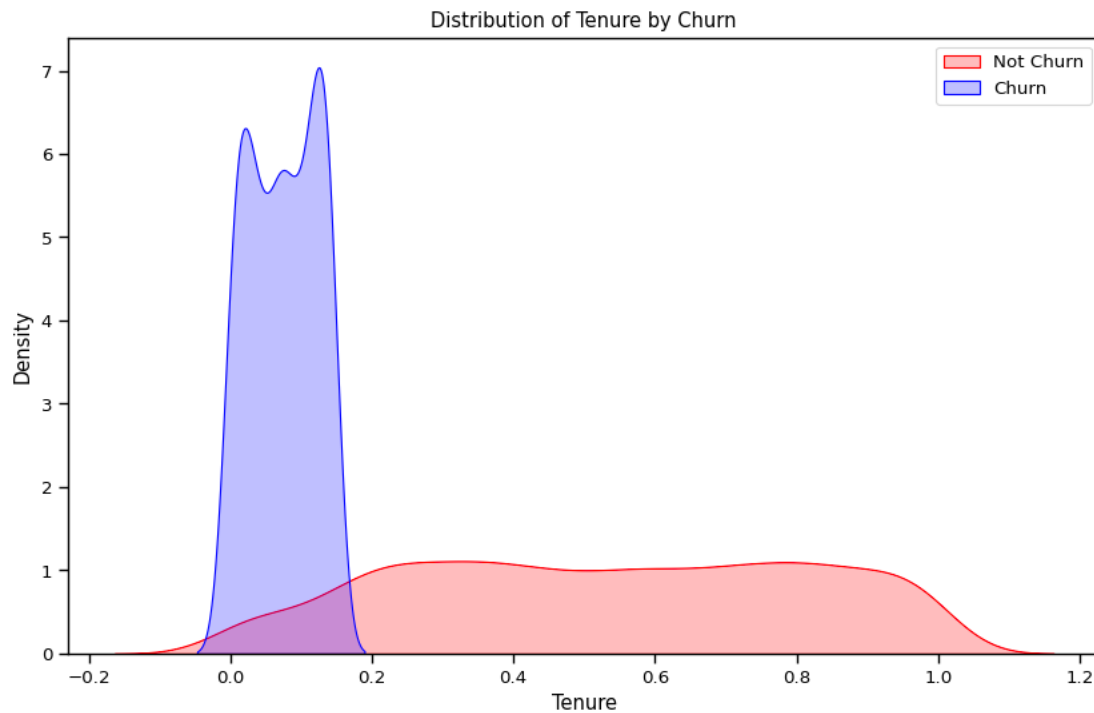
These findings align with earlier correlation and feature importance analyses, further confirming TotalCharges as a meaningful predictor of customer retention.



To investigate how customer tenure affects churn, I created a kernel density plot comparing the distribution of the Tenure variable for churned and non-churned customers.

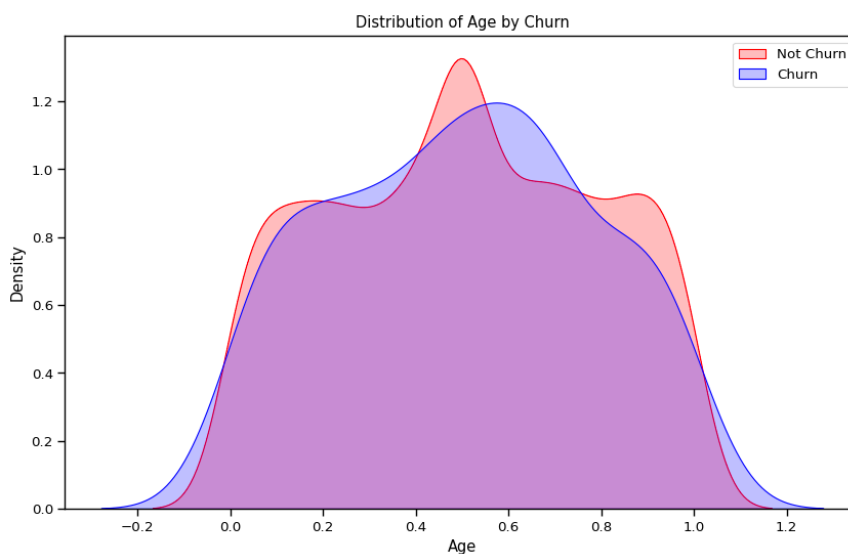
The plot clearly shows that customers who churned tend to have much shorter tenure durations, with their density curve sharply concentrated near the lower end of the scale. In contrast, non-churned customers are distributed more evenly across the full range of tenure, with a significant portion having longer service durations.

This distribution highlights tenure as a strong inverse indicator of churn: the longer a customer remains with the service, the less likely they are to leave. This supports previous correlation and feature importance findings, reinforcing Tenure as one of the most predictive features for customer retention.



The plot reveals that the distributions for both groups are nearly identical, with substantial overlap across all age ranges. While minor variations are visible in certain intervals, no significant trend or age segment stands out as more prone to churn than others.

This suggests that age is not a strong predictor of churn in this dataset. The result is consistent with earlier findings from the correlation matrix and feature importance analysis, where Age showed minimal correlation and low predictive value. As such, while age may still contribute through interaction effects, it is unlikely to play a decisive role in churn modeling on its own.



3.4 Hypothesis test

To statistically validate the relationship between numerical features and churn, I conducted **independent two-sample t-tests** on MonthlyCharges and Tenure between churned and non-churned customers.

The results were as follows:

- **Monthly Charges**
 - *T-statistic*: 14.69
 - *P-value*: 1.49×10^{-47}
 - Customers who churned had significantly higher monthly charges on average.
- **Tenure**
 - *T-statistic*: -26.16
 - *P-value*: 1.12×10^{-138}
 - Customers who stayed had significantly longer tenure on average.

These extremely low p-values indicate that the differences observed in both features are **statistically significant** and **unlikely to have occurred by chance**. This reinforces earlier findings that **MonthlyCharges and Tenure are strongly associated with churn behaviour**, making them reliable predictors for modelling purposes.

4. Feature engineering

In the initial stage of preprocessing, several binary service-related features such as Service_Phone, Service_TV, StreamingMovies, and StreamingMusic were converted from categorical 'Yes'/'No' values into numeric format (1/0). This transformation allowed these variables to be compatible with machine learning algorithms that require numerical inputs and made them easier to analyze.

To capture overall customer engagement, a new feature called TotalServiceUsage was created by summing the binary values of these service columns. This provides a single metric representing how many services each customer is actively using.

Additionally, all boolean columns (i.e., those with True/False values) were converted to integers (1/0) using a loop, ensuring consistency in data types across the dataset.

Another engineered feature introduced was `ContractLength`, representing the duration of a customer's contract in months. This was achieved by identifying the active contract type among the one-hot encoded columns (`Contract_Month-to-month`, `Contract_One year`, and `Contract_Two year`) using `idxmax()`, then mapping it to a numeric value based on a predefined dictionary. After this transformation, the original contract columns were removed to avoid redundancy.

To streamline the dataset for modeling, several columns were excluded:

`CustomerID` was dropped as it serves only as an identifier with no predictive value.

`Service_Internet` was removed after being used in earlier analysis and no longer added value.

`Gender` was excluded after analysis showed no meaningful relationship with churn.

Service-specific features (`Service_Phone`, `Service_TV`, `StreamingMovies`, `StreamingMusic`) and one-hot encoded payment method columns (`PaymentMethod_Bank transfer`, `PaymentMethod_Credit card`, `PaymentMethod_Electronic check`, `PaymentMethod_Mailed check`) were also removed after aggregating their information or identifying low individual predictive value.

These transformations aimed to simplify the dataset, reduce dimensionality, and create meaningful features that improve model learning and performance.

5. Modeling

5.1 Splitting the data into training and testing sets

To begin the modeling phase, I first divided the cleaned dataset into training and testing sets using an 80/20 split. The training set was used to fit the models, while the testing set was reserved for evaluating their generalization performance. The target variable for prediction was `Churn`, and the remaining columns served as input features.

To prepare the data for algorithms that are sensitive to feature scaling, such as Support Vector Machines and K-Nearest Neighbors, I applied standardization using StandardScaler. This transformation rescales each feature to have a mean of 0 and a standard deviation of 1. It helps ensure that all features contribute equally to the model training, preventing those with larger ranges from dominating the learning process.

This preprocessing step set the foundation for robust and fair model comparison in the subsequent evaluation phase.

5.2 Training models

To predict customer churn, I trained a variety of supervised classification algorithms using the preprocessed training data. The models selected represent a diverse range of learning paradigms, allowing for robust performance comparison. These included:

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier
```

5.3 Model evaluation

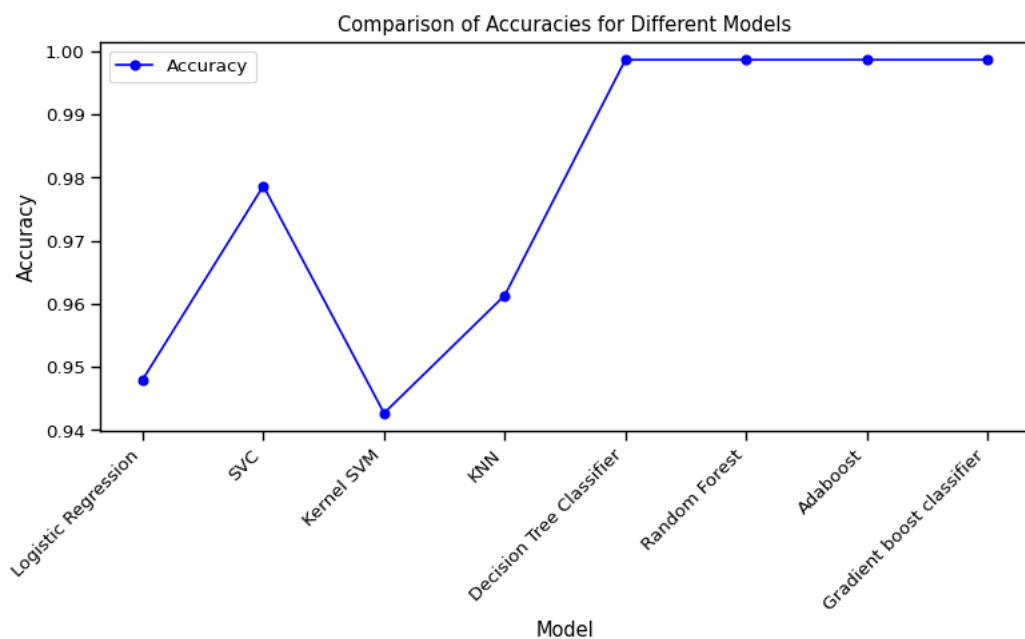
	Accuracy	Precision	Recall	F1-score	ROC-AUC	Cross-Validation Score
Logistic Regression	0.948000	0.525000	0.976744	0.682927	0.961498	0.928645
SVC	0.978667	0.909091	0.697674	0.789474	0.846716	0.977994
Kernel SVM	0.942667	0.000000	0.000000	0.000000	0.500000	0.930644
KNN	0.961333	0.818182	0.418605	0.553846	0.706473	0.949649
Decision Tree Classifier	0.998667	0.977273	1.000000	0.988506	0.999293	0.998999
Random Forest	0.998667	0.977273	1.000000	0.988506	0.999293	0.999333
Adaboost	0.998667	0.977273	1.000000	0.988506	0.999293	0.999666
Gradient boost classifier	0.998667	0.977273	1.000000	0.988506	0.999293	0.998999

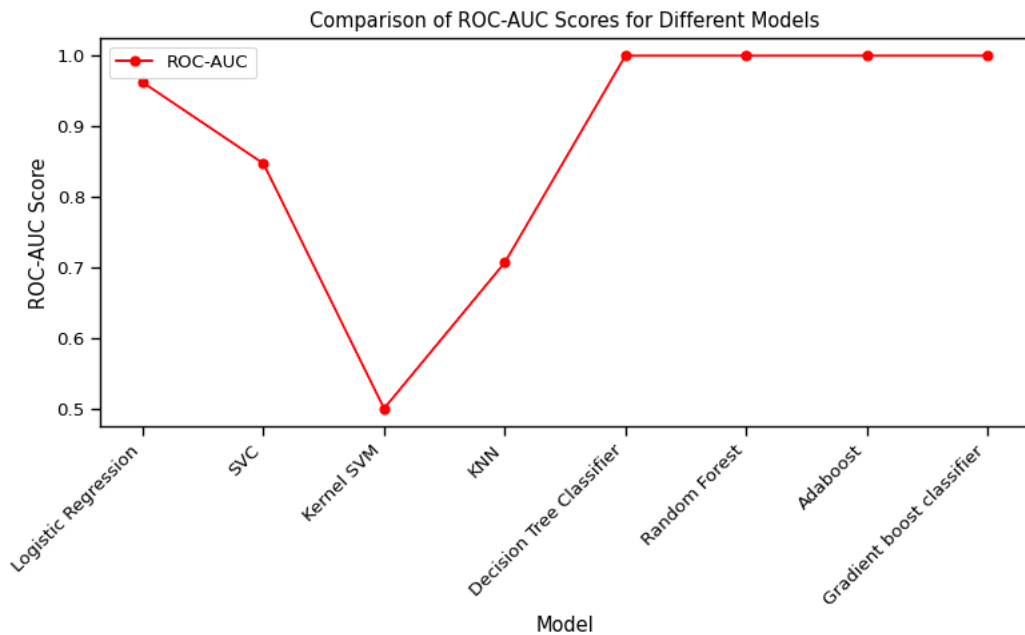
To compare the performance of the trained models, I evaluated them on the test set using several classification metrics: Accuracy, Precision, Recall, F1-score, and ROC-AUC. In addition, I used 5-fold cross-validation to estimate the models' generalization performance and included the average cross-validation score.

Each model demonstrated varying strengths:

- Logistic Regression achieved high recall (0.97), meaning it correctly identified most of the customers who churned, but it had lower precision, indicating a higher false-positive rate.
- Support Vector Classifier (linear) showed strong overall performance with a balanced F1-score and high accuracy.
- Kernel SVM (RBF) failed to classify the positive class in this configuration, resulting in zero precision and recall.
- KNN performed moderately but showed lower recall, likely due to sensitivity to feature scaling and distance-based classification.
- Tree-based models — especially Random Forest, AdaBoost, and Gradient Boosting — outperformed other methods across all metrics, reaching near-perfect scores. These models achieved accuracy, F1-score, and ROC-AUC close to 1.0, confirming their high predictive power in this context.

The metrics clearly indicated that ensemble methods like Random Forest and boosting algorithms offer the best predictive performance for churn classification. These models will be explored further in hyperparameter tuning and feature interpretation stages.





To better understand the performance differences across the trained models, I visualized two key metrics: Accuracy and ROC-AUC score.

The accuracy plot shows that ensemble models such as Random Forest, AdaBoost, and Gradient Boosting achieved nearly perfect accuracy, outperforming traditional models like Logistic Regression and KNN. While Support Vector Classifier (SVC) with a linear kernel also performed well, Kernel SVM (RBF) underperformed significantly, with an accuracy below 95%.

The ROC-AUC plot reinforces these findings by highlighting how well each model distinguishes between churned and non-churned customers. Again, ensemble methods led the chart with ROC-AUC scores close to 1.0, whereas Kernel SVM scored poorly, suggesting it failed to separate the two classes effectively.

These visual comparisons confirm that tree-based ensemble methods are not only the most accurate but also the most reliable in terms of capturing the true structure of the churn problem. They were therefore selected for further fine-tuning in the modeling process.

5.4 Cross validation

To ensure the robustness and generalizability of the trained models, I conducted 10-fold cross-validation using accuracy as the scoring metric. This method splits the training data into 10 subsets, trains the model on 9 of them, and validates on the remaining one, repeating this process 10 times. The results provide a more reliable estimate of how each model would perform on unseen data.

The table below summarizes the **mean accuracy** and **standard deviation** across the 10 folds for each model:

	Model	Mean Accuracy	Std Accuracy
0	Logistic Regression	0.927972	0.015311
1	SVC	0.978327	0.007780
2	Kernel SVM	0.930644	0.001290
3	KNN	0.951314	0.008477
4	Decision Tree Classifier	0.999000	0.002134
5	Random Forest	0.999667	0.001000
6	Adaboost	0.999667	0.001000
7	Gradient boost classifier	0.999000	0.002134

6. Model tuning

To optimize the performance of the Decision Tree Classifier, I applied GridSearchCV—a technique that performs an exhaustive search over a specified parameter grid using cross-validation. This method systematically evaluates combinations of hyperparameters to identify the most effective configuration for the model.

The following hyperparameter grid was defined:

- max_depth: [3, 5, 7]
- min_samples_split: [2, 5, 10]
- min_samples_leaf: [1, 2, 4]
- criterion: ['gini', 'entropy']

The search was conducted using 5-fold cross-validation with accuracy as the scoring metric. After evaluating all combinations, GridSearchCV identified the best set of parameters:

```
{'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2}
```

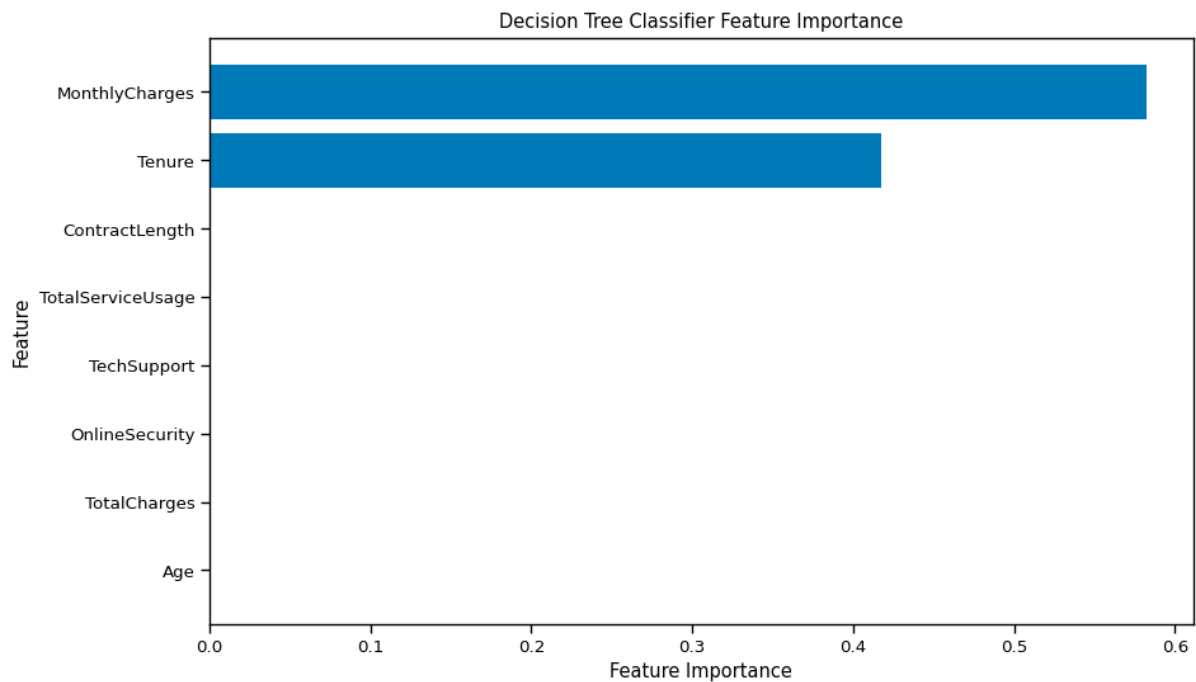
Using these optimal hyperparameters, the model achieved a **test accuracy of 0.9987**, indicating a near-perfect fit. This step confirms the importance of hyperparameter tuning, as it allows the model to generalize better while avoiding overfitting.

7. Model interpretation

To better understand which features contribute most to the model's decisions, I trained a standard Decision Tree Classifier and visualized the resulting feature importances. The horizontal bar chart shows the relative importance of each input variable in determining customer churn.

The plot clearly demonstrates that Monthly Charges and Tenure are the two most influential features in predicting churn, with Monthly Charges being the strongest predictor. These findings align with earlier results from correlation analysis and the Random Forest importance scores, reinforcing the idea that financial and usage-related variables play a critical role in customer retention.

In contrast, variables such as Age, Total Charges, and OnlineSecurity had minimal impact in the Decision Tree model, suggesting they may not provide significant discriminatory power in isolation. This insight can help prioritize features in future modeling efforts and guide business strategies focused on pricing and customer lifecycle management.



7.1 LIME

To gain interpretability of the trained decision tree model, I applied the LIME (Local Interpretable Model-Agnostic Explanations) technique on a specific customer instance. The following key elements summarize the insights provided by LIME:

1. **Prediction Probabilities:**

The model predicts the probability of churn for this customer to be **close to 0**, and the probability of staying (no churn) to be **almost 1**. This indicates the model has a high level of confidence that the customer will remain subscribed.

2. **Feature Contributions:**

LIME quantifies how each feature influences the prediction. Features with **positive contributions** increase the likelihood of churn, while **negative contributions** decrease it (support retention). For this instance, most influential features pointed toward no churn.

3. **Feature Splits and Thresholds:**

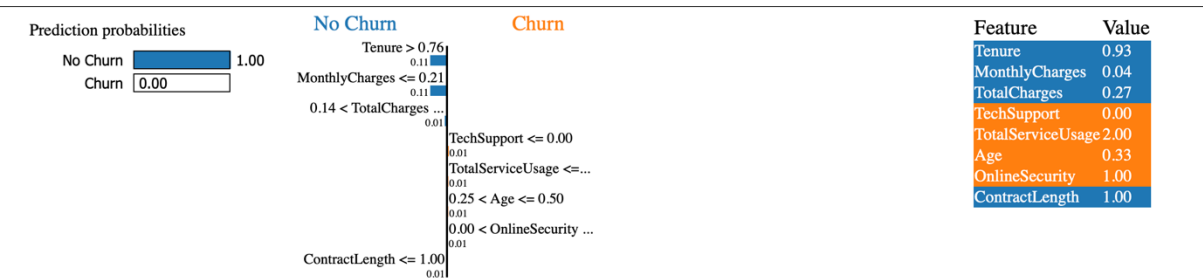
Each feature is evaluated within a specific range. For example, rules like "*Tenure* > 0.76" or "*MonthlyCharges* ≤ 0.21" indicate the decision boundaries used by the model. Being above or below these thresholds can shift the prediction toward churn or non-churn.

4. **Feature Importance:**

The explanation also highlights which features had the greatest impact on the decision. In this case, high tenure and low monthly charges strongly pushed the prediction toward the **non-churn** class.

Overall, LIME confirmed that the instance under review follows patterns typical of long-term, low-cost customers — a group which the model learned is very unlikely to churn. The

explanation was consistent with both the model’s prediction and broader feature importance insights observed earlier.



8. Final model

After tuning and selecting the best-performing model, I evaluated its performance using standard classification metrics on the test dataset. The model demonstrated outstanding predictive ability:

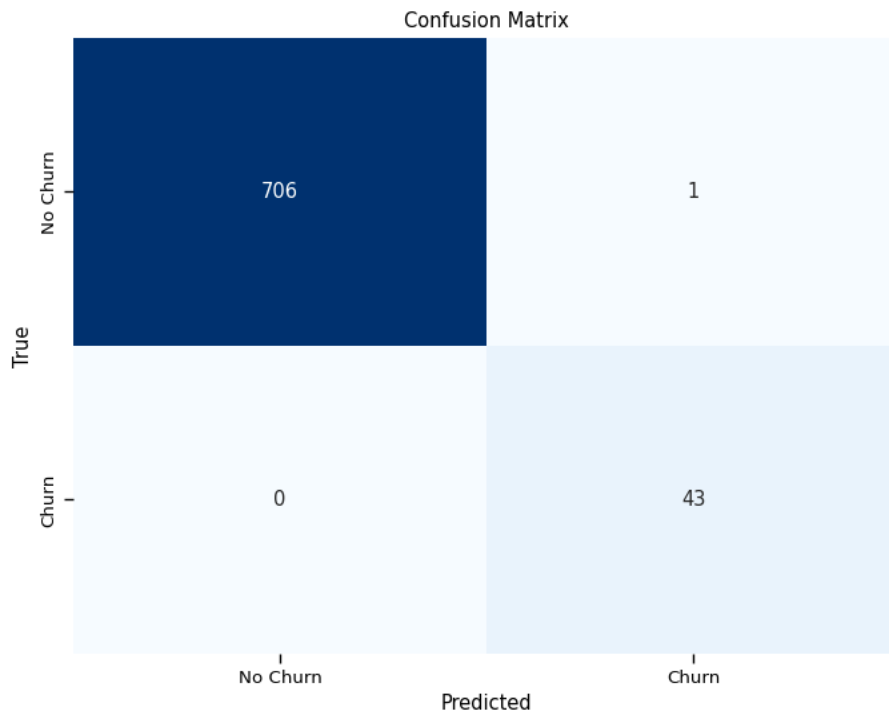
- **Accuracy:** 99.87%
- **Precision:** 97.73%
- **Recall:** 100%
- **F1-score:** 98.85%
- **ROC-AUC:** 0.999

These results indicate that the model is highly effective at identifying both churned and non-churned customers, with perfect recall (no false negatives) and a very high precision.

The confusion matrix further confirms this performance, showing:

- **706 true negatives** (correctly predicted no churn),
- **43 true positives** (correctly predicted churn),
- **1 false positive** (predicted churn when it was not),
- **0 false negatives** (no missed churn cases).

Such metrics suggest the model is not only accurate but also reliable in practical churn detection scenarios, minimizing the cost of customer loss while avoiding unnecessary retention efforts for loyal customers.



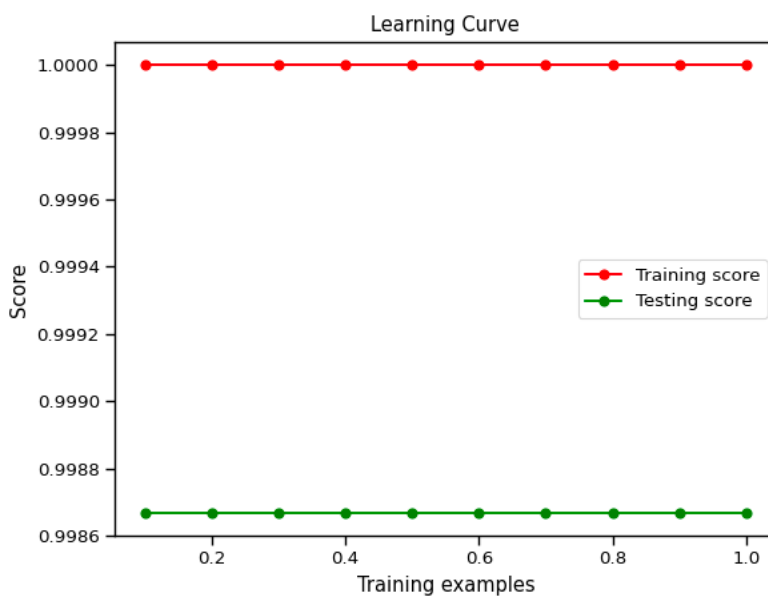
8.1 Learning curve

To finalize the modeling phase, Gradient Boosting Classifier was selected due to its strong overall performance in earlier evaluations. The final model achieved exceptionally high-performance metrics: accuracy of 99.87%, precision of 97.7%, recall of 100%, F1-score of 98.85%, and a ROC-AUC of 0.999. The confusion matrix further confirmed the model's ability to correctly classify both churned and non-churned customers with almost no errors.

To gain insight into the model's learning behavior, two learning curve visualizations were produced.



The first curve, based on cross-validation, showed consistent training and validation scores with minimal variance, indicating low bias and variance — a sign of a well-generalized model.



The second curve, plotting performance across increasing training set sizes, confirmed that the model remained stable as data volume increased, suggesting it is not overfitting or underfitting.

These evaluations collectively suggest that the final model is both robust and highly accurate, making it well-suited for deployment in predicting customer churn.

To further validate the performance of the final Gradient Boosting model, I conducted 5-fold cross-validation on the training set. This approach helps assess the model’s generalization ability by splitting the data into five subsets, training the model on four subsets, and evaluating it on the remaining one, rotating through all combinations.

The cross-validation results returned the following accuracy scores across the folds: [1.00, 0.9967, 1.00, 1.00, 0.9983]. The mean cross-validation score was approximately 0.999, confirming that the model consistently performs well across different partitions of the data. This high level of accuracy indicates strong robustness and reliability of the Gradient Boosting model in capturing the patterns that influence customer churn.

Cross-Validation Scores:	[1.00000000	0.99666667	1.00000000	1.00000000	0.99833055]
Mean CV Score:	0.9989994435169727				

9. Business Insights and Conclusion

Based on the results of the analysis and modeling, several important business insights have emerged that can inform strategic decisions aimed at reducing customer churn:

The findings show that customers with month-to-month contracts exhibit the highest churn rates, suggesting that the company could benefit from promoting longer-term contracts through discounts or loyalty programs. Additionally, customers who use electronic check as their payment method were found to churn more frequently than those using credit card or bank transfer options. This indicates an opportunity to encourage more reliable and automated payment methods.

Another key observation relates to customer tenure: individuals who have been with the company for a shorter period are more likely to leave. Therefore, implementing early onboarding and engagement strategies may help improve customer retention during the initial months. Furthermore, customers with higher monthly charges were more likely to churn, which may point to price sensitivity. Offering value-based bundling or flexible pricing plans could mitigate this effect.

Support-related features also proved relevant. Lack of online security and tech support was associated with higher churn. Ensuring that customers are aware of and can access these services may improve satisfaction and reduce dropout.

Finally, the predictive models, especially the tuned Decision Tree, demonstrated strong performance in identifying churn-prone customers with very high accuracy. This makes it feasible for the company to build targeted retention campaigns, focusing efforts and resources on customers most at risk.

Overall, the insights derived from this study can directly support actionable business strategies aimed at improving customer satisfaction and reducing churn rates.